



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

"МИРЭА - Российский технологический университет"

РТУ МИРЭА

Институт информационных технологий (ИТ)

Кафедра цифровой трансформации (ЦТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №7

по дисциплине

«Разработка баз данных»

Выполнил студент группы ИНБО-20-23

Школьный И.В.

Проверил доцент

Исаева М.В.

Москва 2025

ПРАКТИЧЕСКАЯ РАБОТА №7. ОПТИМИЗАЦИЯ ЗАПРОСОВ И УПРАВЛЕНИЕ ТРАНЗАКЦИЯМИ В POSTGRESQL

Цель работы:

Целью данной практической работы является формирование у студентов практических навыков анализа и оптимизации производительности SQL-запросов, а также освоение механизмов управления транзакциями для обеспечения целостности данных (согласно принципам ACID) в СУБД PostgreSQL.

Постановка задачи:

Для выполнения практической работы необходимо последовательно выполнить следующие шаги, адаптируя примеры из БД «Аптека» к вашей собственной базе данных: Подготовка базы данных. Следуя руководству, наполнить одну из ключевых таблиц вашей БД большим объемом данных (не менее 20 000 строк). Это обязательно для корректной демонстрации работы оптимизатора.

Задание №1: анализ и оптимизация (3 сценария)

Определить три различных «медленных» запроса к вашей БД, которые можно оптимизировать с помощью разных типов индексов (например, стандартный B-Tree, индекс по выражению, частичный/отфильтрованный индекс).

Если в вашей базе недостаточно данных, выполните для одной из своих таблиц действия по автоматической генерации содержимого, описанные в разделе «Подготовка базы данных».

Для каждого из 3-х сценариев:

1. Выполнить анализ запроса «КАК ЕСТЬ» (без индекса) с помощью EXPLAIN ANALYZE.
2. Привести план выполнения «ДО», письменно проанализировать его и выявить причину низкой производительности (например, Seq Scan).
3. Создать необходимый INDEX для оптимизации.

4. Повторно выполнить EXPLAIN ANALYZE.

5. Привести план выполнения «ПОСЛЕ», демонстрирующий использование индекса (например, Index Scan).

6. Обязательно сформировать сравнительную таблицу, демонстрирующую разницу в производительности (план, Execution Time) «ДО» и «ПОСЛЕ».

Задание №2: демонстрация атомарной транзакции (COMMIT).

По примеру раздела 2.2 реализуйте в своей базе одну бизнес-операцию (минимум две связанные операции изменения данных) внутри транзакции BEGIN...COMMIT.

Задokumentировать все шаги и результаты, сделать выводы.

Задание №3: демонстрация отката транзакции (ROLLBACK).

Адаптировать приведённый в разделе 2.3 SQL-скрипт, моделирующий сбой операции, под свою предметную область, повторив описанные действия. Задokumentировать все шаги и результаты, сделать выводы.

Задание №4: моделирование аномалии «Неповторяемое чтение».

Используя два редактора SQL, смоделировать проблему «неповторяемое чтение» на уровне изоляции по умолчанию (READ COMMITTED) по приведённому в разделе 2.4 образцу.

Задokumentировать все шаги и результаты, сделать выводы.

Задание №5: устранение аномалии «Неповторяемое чтение».

Повторить моделирование из Задания №4, но с использованием уровня изоляции REPEATABLE READ. В качестве образца использовать раздел 2.5.

Задokumentировать все шаги и результаты, сделать выводы.

ХОД ВЫПОЛНЕНИЯ РАБОТЫ

Для создания понятных и последовательных примеров в рамках данной работы будет использоваться упрощенная схема базы данных «Гостиницы». Эта схема послужит основой для демонстрации работы триггеров и курсоров. Студентам следует адаптировать представленные примеры для своих собственных баз данных.

Ниже приводятся таблицы, используемые для построения запросов.

Таблица 1. Таблица Бронирование (Booking)

Auto

postgresshkolnyi_iv@dbstud

<postgres> pr7

booking

СвойстваДанныеДиаграмма

Показать SQL

Введите SQL выражение чтобы отфильтровать результаты

	123 id	123 guest_id	123 room_id	check_in_date	check_out_date	123 loyalty_id
1	1	1	1	2023-06-01	2023-06-10	1
2	2	2	2	2023-07-15	2023-07-20	2
3	3	3	3	2023-08-05	2023-08-15	3
4	4	4	4	2023-09-01	2023-09-05	1
5	5	5	5	2023-09-10	2023-09-15	2
6	6	6	6	2023-09-20	2023-09-25	3
7	7	7	7	2023-10-01	2023-10-07	1
8	8	8	8	2023-10-15	2023-10-20	2
9	9	9	9	2023-11-05	2023-11-12	3
10	10	10	10	2023-11-20	2023-11-25	1
11	11	1	1	2024-01-15	2024-01-20	1
12	12	1	1	2024-02-01	2024-02-04	1

Таблица 2. Таблица Документы о бронировании (Booking_document)

Auto postgres shkolnyi_iv@dbstud

<postgres> pr7 booking_document

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

Таблица	123 id	123 booking_id	issue_date	A-Z status
1	1	1	2023-05-15	Confirmed
2	2	2	2023-06-20	Paid
3	3	3	2023-07-10	Completed
4	4	4	2023-08-15	Confirmed
5	5	5	2023-08-25	Paid
6	6	6	2023-09-10	Completed
7	7	7	2023-09-20	Confirmed
8	8	8	2023-10-05	Paid
9	9	9	2023-10-25	Completed
10	10	10	2023-11-10	Confirmed
11	11	11	2025-10-27	Confirmed
12	12	12	2024-01-20	Confirmed

Таблица 3. Таблица Брошюра (Brochure)

Auto postgres shkolnyi_iv@dbstud

brochure

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

Таблица	123 id	123 hotel_id	A-Z language	publication_date
1	1	1	Russian	2022-01-10
2	2	2	English	2022-03-15
3	3	3	German	2022-05-20
4	4	1	English	2022-02-28
5	5	2	German	2022-04-10
6	6	3	Russian	2022-06-15
7	7	1	German	2022-07-20
8	8	2	English	2022-08-25
9	9	3	English	2022-09-30
10	10	1	Russian	2022-10-05
11	11	2	Russian	2022-11-12
12	12	3	German	2022-12-18

Таблица 4. Таблица Регистрация (Check_in)

	123 id	123 guest_id	123 total_bill	123 room_id	123 doc_id
1	1	1	45 000	1	1
2	2	2	42 500	2	2
3	3	3	120 000	3	3
4	4	4	18 000	4	4
5	5	5	39 000	5	5
6	6	6	57 500	6	6
7	7	7	31 200	7	7
8	8	8	41 000	8	8
9	9	9	84 000	9	9
10	10	10	27 500	10	10
11	11	1	12 300	1	12

Таблица 5. Таблица Работник (Employee)

	123 id	A-Z full_name	123 position_id	hire_date	123 hotel_id
1	1	Смирнова Ольга Дмитриевна	1	2018-06-10	1
2	2	Кузнецов Андрей Петрович	2	2020-02-15	2
3	3	Васильева Елена Ивановна	3	2021-05-20	3
4	4	Петров Максим Игоревич	4	2019-03-12	1
5	5	Семенова Ирина Васильевна	5	2020-08-25	2
6	6	Ковалев Александр Петрович	6	2018-11-30	3
7	7	Новикова Юлия Сергеевна	7	2021-02-14	1
8	8	Данилов Алексей Викторович	8	2017-07-22	2
9	9	Тихонова Марина Олеговна	9	2016-05-18	3
10	10	Фролов Денис Николаевич	10	2022-01-10	1

Таблица 6. Таблица Гость (Guest)

Auto postgres shkolnyi_iv@dbstud

guest X

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

Таблица	123 id	A-Z full_name	A-Z phone_number	A-Z email
1	1	Козлова Мария Игоревна	7911123456	kozlova@mail.ru
2	2	Николаев Дмитрий Олегович	7902876543	nikolaev@gmail.com
3	3	Федорова Екатерина Павловна	7922555777	fedorova@yandex.ru
4	4	Громов Сергей Александрович	7913987654	gromov@mail.ru
5	5	Волкова Анастасия Викторовна	7904765432	volkova@gmail.com
6	6	Белов Артем Сергеевич	7923876541	belov@yandex.ru
7	7	Соколова Ольга Николаевна	7915234567	sokolova@mail.ru
8	8	Иванов Иван Иванович	7910123456	ivanov@mail.ru
9	9	Петрова Анна Сергеевна	7909876543	petrova@gmail.com
10	10	Сидоров Алексей Викторович	7921555777	sidorov@yandex.ru

Таблица 7. Таблица Документы гостя (Guest_document)

uto postgres shkolnyi_iv@dbstud

guest_document X

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

Таблица	123 id	123 guest_id	A-Z document_data	A-Z document_type	issue_date	expiry_date
1	1	1	1234567890	Passport	2015-05-20	2025-05-20
2	2	2	0987654321	Birth certificate	2020-01-15	2030-01-15
3	3	3	1122334455	Driver license	2018-11-30	2028-11-30
4	4	4	4455667788	Passport	2017-03-10	2027-03-10
5	5	5	5566778899	Driver license	2019-08-22	2029-08-22
6	6	6	6677889900	Passport	2016-11-05	2026-11-05
7	7	7	7788990011	Birth certificate	2021-02-14	2031-02-14
8	8	8	8899001122	Driver license	2018-07-30	2028-07-30
9	9	9	9900112233	Passport	2019-12-20	2029-12-20
10	10	10	1011223344	Birth certificate	2020-04-18	2030-04-18

Таблица 8. Таблица Гостиница (Hotel)

uto postgres shkolnyi_iv@dbstud

guest_document hotel X

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

Таблица	123 id	A-Z name	A-Z address	123 star_rating	A-Z description
1	1	City Lights	ул. Невский проспект, 45	4	Современный отель в историческом центре
2	2	River Side	наб. реки Мойки, 12	5	Элитный отель с видом на реку
3	3	Park Inn	ул. Садовая, 78	3	Комфортабельный отель рядом с парком

Таблица 9. Таблица Программа лояльности (Loyalty_program)

auto postgres shkolnyi_iv@dbstud

loyalty_program

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

123 id	A-Z name	123 discount	A-Z description
1	Gold Member	15	Exclusive discounts and early check-in for gold members
2	Silver Member	10	Special offers and room upgrades for silver members
3	Bronze Member	5	Basic discounts for frequent guests

Таблица 10. Таблица Должность (Position)

auto postgres shkolnyi_iv@dbstud

loyalty_program position

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

123 id	A-Z title	123 base_rate
1	Менеджер	50 000
2	Портье	35 000
3	Горничная	30 000
4	Администратор	45 000
5	Шеф-повар	80 000
6	Официант	32 000
7	Охранник	38 000
8	Уборщик	28 000
9	Бухгалтер	55 000
10	Маркетолог	48 000
11	IT-специалист	60 000
12	Директор	100 000
13	Аниматор	35 000

Таблица 11. Таблица Номер (Room)

auto

postgres

shkolnyi_iv@dbstud

loyalty_program

position

room X

Свойства

Данные

Диаграмма

Показать SQL | Введите SQL выражение чтобы отфильтровать результаты

Таблица	123 id	A-Z room_number	A-Z type	123 price_per_night	123 hotel_id
1	1	101	Standard	5 000	1
2	2	202	Deluxe	8 500	2
3	3	301	Suite	12 000	3
4	4	102	Standard	4 500	1
5	5	103	Deluxe	7 500	1
6	6	201	Standard	4 800	2
7	7	202	Deluxe	7 800	2
8	8	203	Suite	11 500	2
9	9	301	Standard	5 200	3
10	10	302	Deluxe	8 200	3
11	11	303	Suite	12 500	3
12	12	401	Standard	5 500	1

Текст

Подготовка данных

Нам нужно наполнить одну из таблиц 20000+ записями. Для демонстрации оптимизации лучше всего подойдет таблица Room.

Листинг 1. Заполнение таблицы room 20000 записями

```
DO $$
DECLARE
  i INT;
  room_types TEXT[] := ARRAY['Standard', 'Deluxe', 'Suite'];
  base_prices DECIMAL[] := ARRAY[4000, 8000, 12000]; -- Базовые цены для каждого
типа
BEGIN
  FOR i IN 1..20000 LOOP
    INSERT INTO Room (room_number, type, price_per_night, hotel_id)
    VALUES (
      -- Генерируем номер комнаты (от 403 до )
      ((402 + (i % 6667))::VARCHAR),
      -- Случайный тип комнаты из трех доступных
      room_types[(random() * 2 + 1)::INT],
      -- Цена: базовая цена + случайное отклонение ±20%
      (base_prices[(random() * 2 + 1)::INT] * (0.8 + random() * 0.4))::DECIMAL(10,2),
      -- Случайный отель из 3 существующих
      (random() * 2 + 1)::INT
    );

    -- Логируем прогресс каждые 2000 записей
    IF i % 2000 = 0 THEN
      RAISE NOTICE 'Добавлено % номеров', i;
    END IF;
  END LOOP;
END;
$$ LANGUAGE plpgsql;
```

Запустим скрипт.

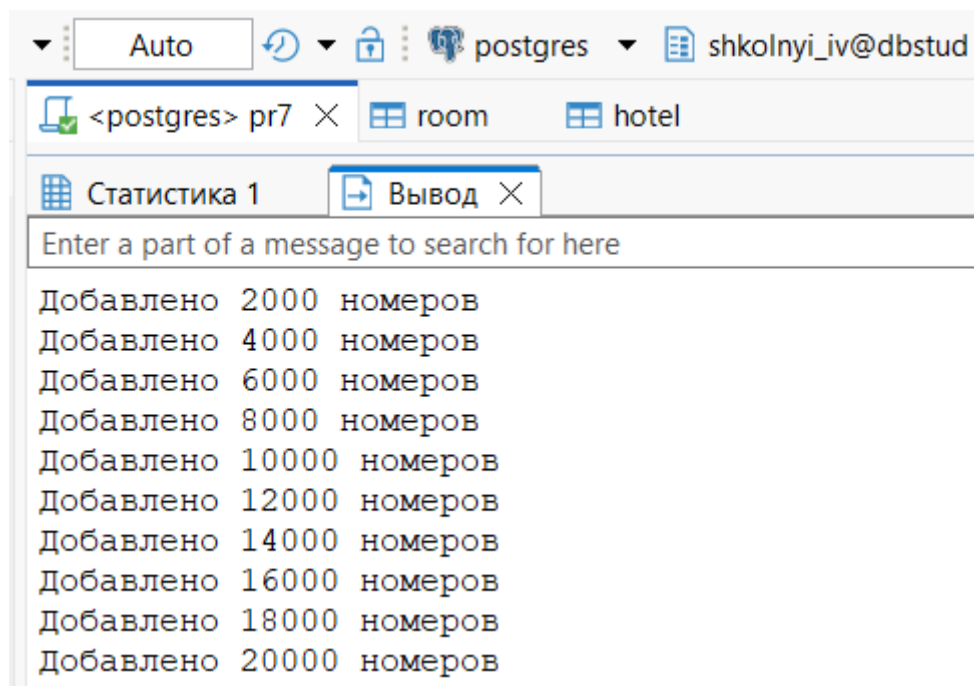


Рисунок 1 — Генерация номеров

Задание 1. Анализ и оптимизация (3 сценария)

Сценарий 1. B-Tree индекс для поиска номеров по типу.

Выполним первый шаг. Вызовем анализ без индекса.

Листинг 2. Анализ без индекса

```
EXPLAIN ANALYZE
SELECT * FROM Room
WHERE type = 'Deluxe';
```

Запустим скрипт.

Результат 1 X	
DO \$\$ DECLARE i INT; room_types TEXT[] := AR; Введите SQL выражение чтобы отфильтровать результаты	
Таблица	A-Z QUERY PLAN
1	Seq Scan on room (cost=0.00..403.15 rows=10034 width=31) (actual time=0.035..4.123 rows=10034 loops=1)
2	Filter: ((type)::text = 'Deluxe'::text)
3	Rows Removed by Filter: 9978
4	Planning Time: 0.605 ms
5	Execution Time: 4.608 ms

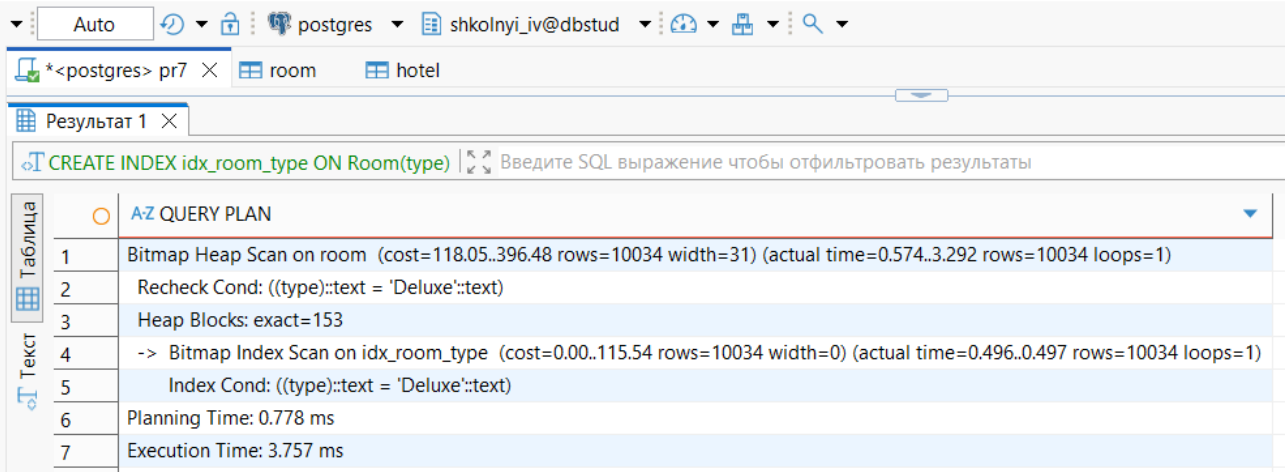
Рисунок 2 — Анализ до применения индексов

Создадим B-Tree индекс по типу номера.

Листинг 3. Создание B-Tree индекса

```
CREATE INDEX idx_room_type ON Room(type);
```

Индекс успешно создан, теперь повторно выполним изначальный запрос.



Auto	postgres	shkolnyi_iv@dbstud
*<postgres> pr7	room	hotel
Результат 1		
CREATE INDEX idx_room_type ON Room(type)		
Введите SQL выражение чтобы отфильтровать результаты		
Аз QUERY PLAN		
1	Bitmap Heap Scan on room	(cost=118.05..396.48 rows=10034 width=31) (actual time=0.574..3.292 rows=10034 loops=1)
2	Recheck Cond: ((type)::text = 'Deluxe'::text)	
3	Heap Blocks: exact=153	
4	-> Bitmap Index Scan on idx_room_type	(cost=0.00..115.54 rows=10034 width=0) (actual time=0.496..0.497 rows=10034 loops=1)
5	Index Cond: ((type)::text = 'Deluxe'::text)	
6	Planning Time: 0.778 ms	
7	Execution Time: 3.757 ms	

Рисунок 3 — Анализ после применения индексов

Вместо последовательного сканирования каждой строки, используется сканирование Bitmap Heap, в котором было просмотрено 153 блока.

Таблица 1. Сравнительная таблица

Анализ до	Анализ после
Seq scan	Beatmap heap scan
4.608 ms	3.757 ms

Сценарий 2. Индекс по выражению для поиска номеров по цене (с округлением)

Выполним первый шаг. Вызовем анализ без индекса.

Листинг 4. Анализ без индекса

```
EXPLAIN ANALYZE
SELECT * FROM Room
WHERE ROUND(price_per_night) = 10000;
```

Запустим скрипт.

Auto	
postgres shkolnyi_iv@dbstud	
*<postgres> pr7 × room hotel	
Результат 1 ×	
EXPLAIN ANALYZE SELECT * FROM Room WHERE Введите SQL выражение чтобы отфильтровать результаты	
Таблица Текст	A-Z QUERY PLAN
	1 Seq Scan on room (cost=0.00..453.18 rows=100 width=31) (actual time=2.588..4.998 rows=1 loops=1)
	2 Filter: (round(price_per_night) = '10000'::double precision)
	3 Rows Removed by Filter: 20011
	4 Planning Time: 1.325 ms
	5 Execution Time: 5.093 ms

Рисунок 4 — Анализ до применения индексов

Создадим индекс.

Листинг 5. Создание индекса по выражению

```
CREATE INDEX idx_room_price_rounded ON Room (ROUND(price_per_night));
```

Индекс успешно создан, теперь повторно выполним изначальный запрос.

Auto	
postgres shkolnyi_iv@dbstud	
*<postgres> pr7 × room hotel	
Результат 1 ×	
CREATE INDEX idx_room_price_rounded ON Ro Введите SQL выражение чтобы отфильтровать результаты	
Таблица Текст	A-Z QUERY PLAN
	1 Bitmap Heap Scan on room (cost=5.06..149.87 rows=100 width=31) (actual time=0.074..0.075 rows=1 loops=1)
	2 Recheck Cond: (round(price_per_night) = '10000'::double precision)
	3 Heap Blocks: exact=1
	4 -> Bitmap Index Scan on idx_room_price_rounded (cost=0.00..5.04 rows=100 width=0) (actual time=0.051..0.051 rows=1 loops=1)
	5 Index Cond: (round(price_per_night) = '10000'::double precision)
	6 Planning Time: 0.657 ms
	7 Execution Time: 0.115 ms

Рисунок 5 — Анализ с индексом

Как видим, запрос работает быстрее с применением индекса.

Таблица 2. Сравнительная таблица

Анализ до	Анализ после
Seq scan	Bitmap heap scan
5.093 ms	0.115 ms

Сценарий 3. Частичный индекс для дорогих номеров

Выполним первый шаг. Вызовем анализ без индекса.

Листинг 6. Анализ без индекса

```
EXPLAIN ANALYZE
SELECT * FROM Room
WHERE price_per_night > 15000;
```

Запустим скрипт.

A-Z QUERY PLAN	
1	Seq Scan on room (cost=0.00..403.15 rows=2 width=31) (actual time=4.647..4.648 rows=0 loops=1)
2	Filter: (price_per_night > '15000'::double precision)
3	Rows Removed by Filter: 20012
4	Planning Time: 1.237 ms
5	Execution Time: 4.723 ms

Рисунок 6 — Анализ до применения индексов

Создадим индекс.

Листинг 7. Создание частичного индекса

```
CREATE INDEX idx_room_expensive ON Room(id)
WHERE price_per_night > 15000;
```

Индекс успешно создан, теперь повторно выполним изначальный запрос.

A-Z QUERY PLAN	
1	Index Scan using idx_room_expensive on room (cost=0.12..8.15 rows=2 width=31) (actual time=0.004..0.004 rows=0 loops=1)
2	Planning Time: 0.707 ms
3	Execution Time: 0.062 ms

Рисунок 7 — Анализ с индексом

Как видим, запрос работает быстрее с применением индекса.

Таблица 3. Сравнительная таблица

Анализ до	Анализ после
Seq scan	Index scan
4.723 ms	0.062 ms

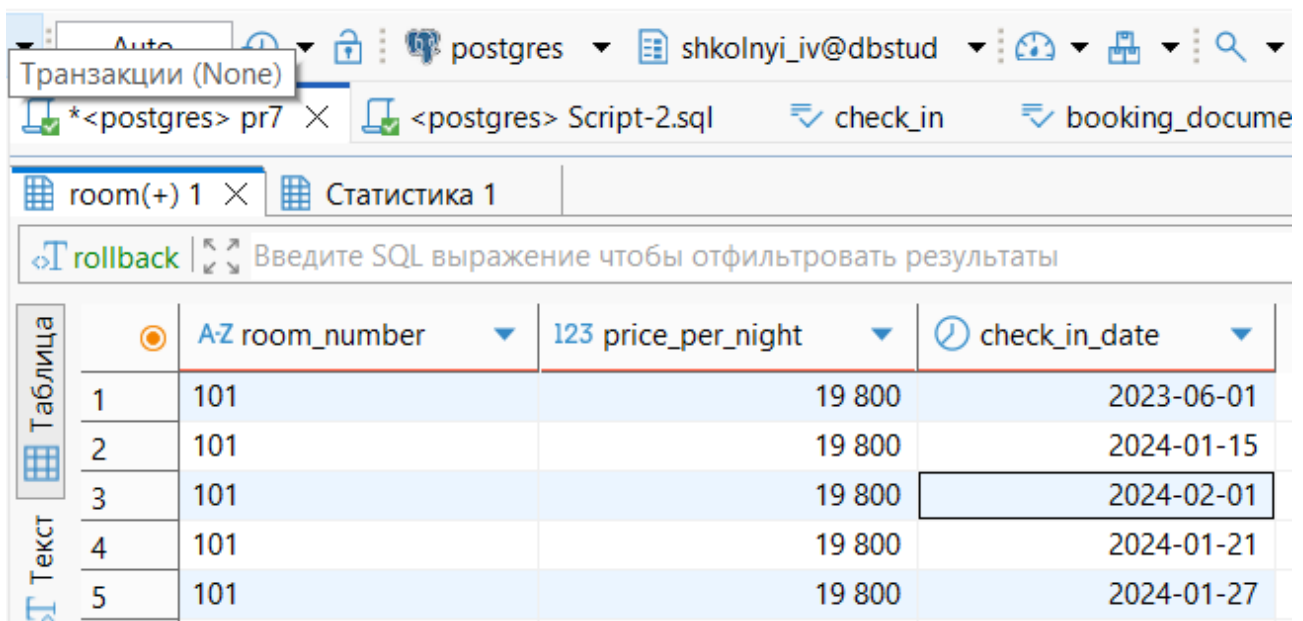
Задание 2. Демонстрация атомарной транзакции (commit)

Бизнес-операция: Обновление цены номера и создание бронирования.

Листинг 8. Атомарная транзакция

```
BEGIN;  
-- 1. Создаем бронирование по новой цене  
INSERT INTO Booking (guest_id, room_id, check_in_date, check_out_date, loyalty_id)  
VALUES (1, 1, '2024-01-27', '2024-01-30', 1);  
  
--2. Добавляем документ о бронировании  
INSERT INTO booking_document (booking_id, issue_date, status)  
VALUES (currval('booking_id_seq'), '2024-01-27', 'Confirmed');  
--3. Фиксируем регистрацию  
INSERT INTO check_in (guest_id, total_bill, room_id, doc_id)  
VALUES (1, 0, 1, currval('booking_document_id_seq'));  
COMMIT;  
  
-- Проверяем результат  
SELECT r.room_number, r.price_per_night, b.check_in_date  
FROM Room r  
JOIN Booking b ON r.id = b.room_id  
WHERE r.id = 1;
```

Запустим скрипт.



The screenshot shows a PostgreSQL client window with a table named 'room(+)' containing 5 rows. The table has columns: 'room_number', 'price_per_night', and 'check_in_date'. The first row is highlighted. The interface also shows a 'rollback' button and a search bar.

Таблица	room_number	price_per_night	check_in_date
1	101	19 800	2023-06-01
2	101	19 800	2024-01-15
3	101	19 800	2024-02-01
4	101	19 800	2024-01-21
5	101	19 800	2024-01-27

Рисунок 8 — Результат

Задание 3. Демонстрация отката транзакции (rollback)

Моделирование сбоя при обновлении номеров.

Листинг 9. Откат транзакции

```
BEGIN;  
  
INSERT INTO Booking (guest_id, room_id, check_in_date, check_out_date, loyalty_id)
```

```
VALUES (1, 1, '2024-01-27', '2024-01-30', 1);
```

--2. Добавляем документ о бронировании

```
INSERT INTO booking_document (booking_id, issue_date, status)
```

```
VALUES (10, '2024-01-20', 'Confirmed');
```

--3. Фиксируем регистрацию

```
INSERT INTO check_in (guest_id, total_bill, room_id, doc_id)
```

```
VALUES (1, 0, 1, currval('booking_document_id_seq'));
```

```
commit;
```

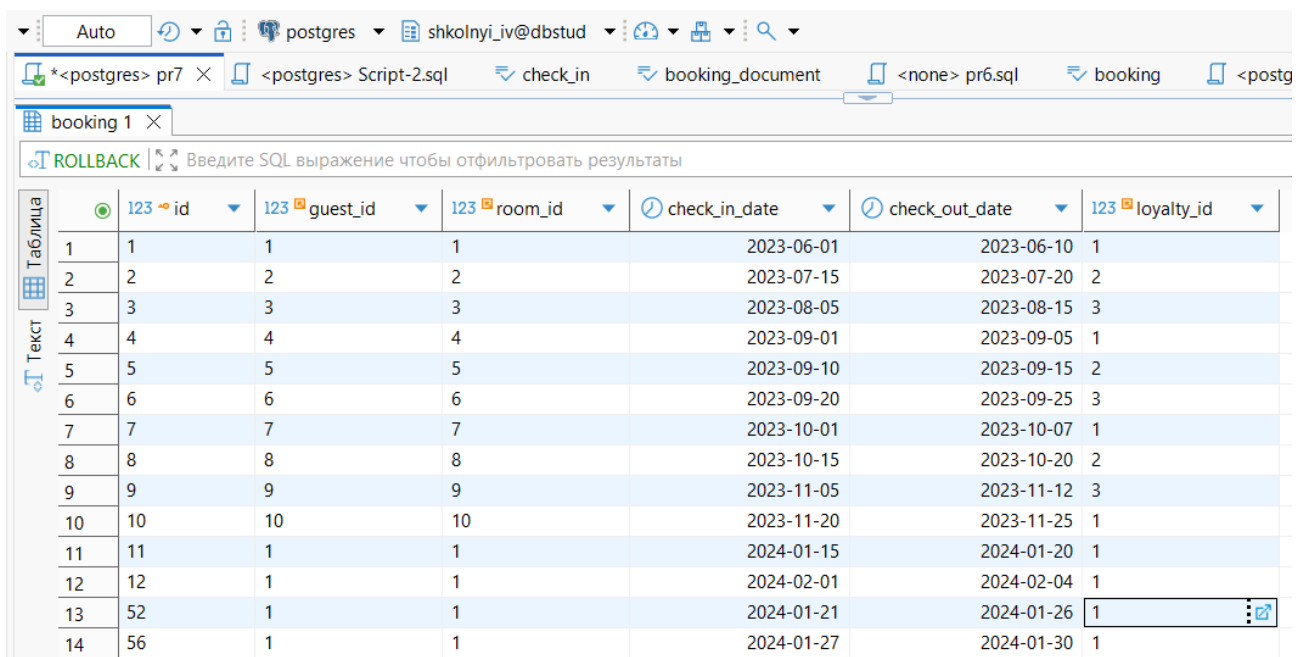
```
select * from booking;
```

-- После ошибки вручную выполняем:

```
ROLLBACK;
```

```
select * from booking;
```

Так выглядит таблица Booking до неправильной транзакции.



	id	guest_id	room_id	check_in_date	check_out_date	loyalty_id
1	1	1	1	2023-06-01	2023-06-10	1
2	2	2	2	2023-07-15	2023-07-20	2
3	3	3	3	2023-08-05	2023-08-15	3
4	4	4	4	2023-09-01	2023-09-05	1
5	5	5	5	2023-09-10	2023-09-15	2
6	6	6	6	2023-09-20	2023-09-25	3
7	7	7	7	2023-10-01	2023-10-07	1
8	8	8	8	2023-10-15	2023-10-20	2
9	9	9	9	2023-11-05	2023-11-12	3
10	10	10	10	2023-11-20	2023-11-25	1
11	11	1	1	2024-01-15	2024-01-20	1
12	12	1	1	2024-02-01	2024-02-04	1
13	52	1	1	2024-01-21	2024-01-26	1
14	56	1	1	2024-01-27	2024-01-30	1

Рисунок 9 — Booking до транзакции

Запускаем скрипт.

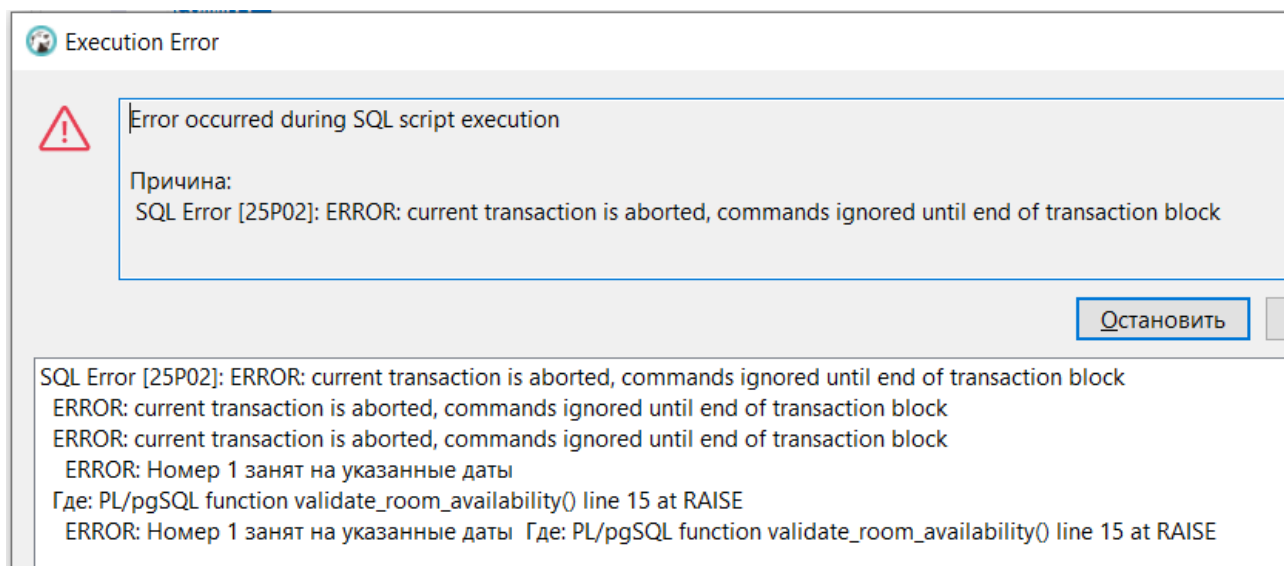


Рисунок 10 — Ошибка запуска

Если попробовать обратиться к таблице Booking, то тоже будет ошибка.

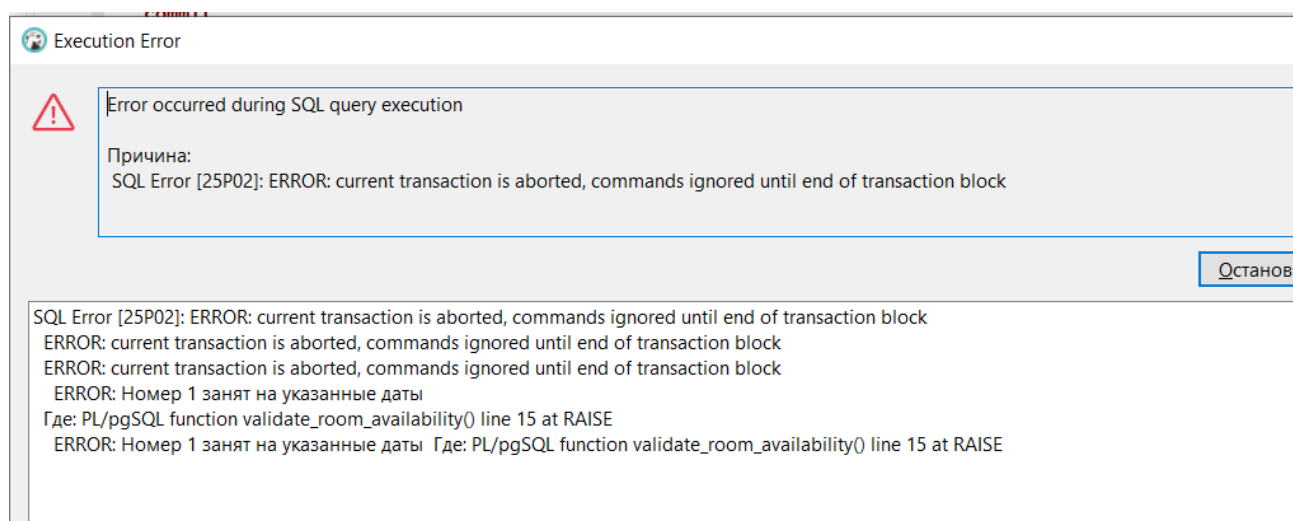


Рисунок 11 — Обращение к booking

Выполняем rollback и наблюдаем отсутствие изменений в таблице.

	id	guest_id	room_id	check_in_date	check_out_date	loyalty_id
1	1	1	1	2023-06-01	2023-06-10	1
2	2	2	2	2023-07-15	2023-07-20	2
3	3	3	3	2023-08-05	2023-08-15	3
4	4	4	4	2023-09-01	2023-09-05	1
5	5	5	5	2023-09-10	2023-09-15	2
6	6	6	6	2023-09-20	2023-09-25	3
7	7	7	7	2023-10-01	2023-10-07	1
8	8	8	8	2023-10-15	2023-10-20	2
9	9	9	9	2023-11-05	2023-11-12	3
10	10	10	10	2023-11-20	2023-11-25	1
11	11	1	1	2024-01-15	2024-01-20	1
12	12	1	1	2024-02-01	2024-02-04	1
13	52	1	1	2024-01-21	2024-01-26	1
14	56	1	1	2024-01-27	2024-01-30	1

Рисунок 12 — Изначальная таблица Booking

Задание 4. Моделирование аномалии неповторяемое чтение

Напишем два скрипта. В одном будет выполнять select номера 101, в другом будем обновлять стоимость этого номера.

Листинг 10. Скрипт 1

```
BEGIN;
-- Уровень изоляции по умолчанию: READ COMMITTED

-- 1. Первое чтение цены номера
SELECT room_number, price_per_night FROM Room WHERE id = 1;

-- 2. Пауза 10 секунд
SELECT pg_sleep(10);

-- 3. Второе чтение (увидит изменения из Скрипта Б)
SELECT room_number, price_per_night FROM Room WHERE id = 1;

COMMIT;
```

Листинг 11. Скрипт 2

```
-- Повышаем цену номера
UPDATE Room SET price_per_night = price_per_night * 2 WHERE id = 1;
COMMIT;
```

Запускаем скрипты.

Таблица	room_number	price_per_night
1	101	6 600

Рисунок 10 — Select до обновления

Таблица	room_number	price_per_night
1	101	13 200

Рисунок 11 — Select после обновления

Задание 5. Устранение аномалии «Неповторяемое чтение»

Откатим назад стоимость номера и идентификатором 1 на 6600. В этом задании необходимо доказать, что repeatable read справляется с аномалией и не выводит два разных результата.

Листинг 12. Скрипт 1

```
BEGIN;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

-- 1. Первое чтение цены номера
SELECT room_number, price_per_night FROM Room WHERE id = 1;

-- 2. Пауза 10 секунд
SELECT pg_sleep(10);

-- 3. Второе чтение (НЕ увидит изменения из Скрипта Б)
SELECT room_number, price_per_night FROM Room WHERE id = 1;

COMMIT;
```

Листинг 13. Скрипт 2

```
-- Повышаем цену номера
```

```
UPDATE Room SET price_per_night = price_per_night * 3 WHERE id = 1;  
COMMIT;
```

Запускаем скрипты.

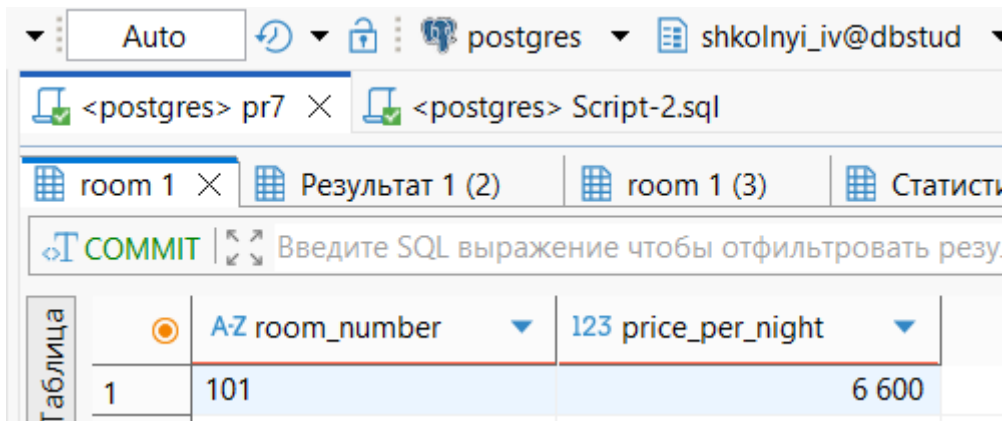


Таблица	A-Z room_number	123 price_per_night
1	101	6 600

Рисунок 12 — Select до обновления

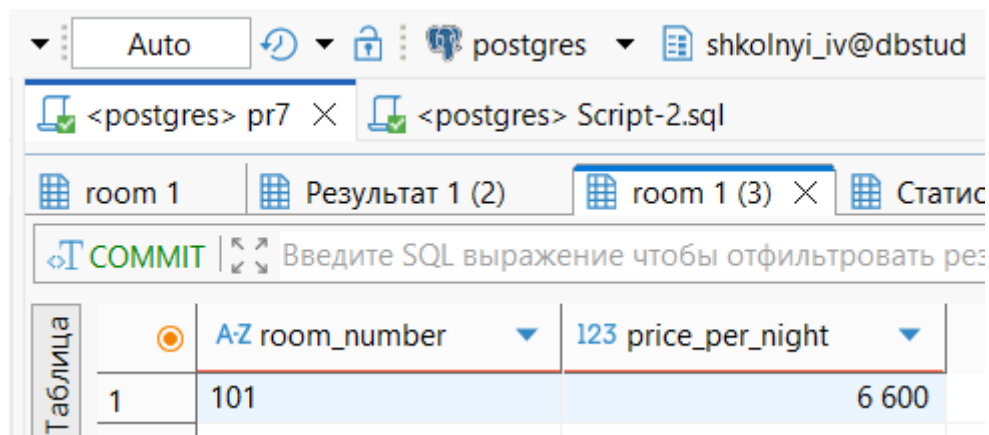


Таблица	A-Z room_number	123 price_per_night
1	101	6 600

Рисунок 13 — Select после обновления

Как видим, repeatable read успешно справился.

ВЫВОД

В ходе выполнения практической работы были освоены навыки анализа и оптимизации производительности SQL-запросов, а также освоение механизмов управления транзакциями для обеспечения целостности данных (согласно принципам ACID) в СУБД PostgreSQL.